

Project name

MedAssist - Offline AI Clinical Triage Assistant

My team

Abdul Rafay Imran - Solo Founder and Technical Lead

End-to-end ownership of MedAssist as an independent participant, including system architecture, FastAPI backend, React + Tailwind frontend, MedGemma Colab runtime setup, prompt engineering, evaluation, safety controls, documentation, and demo video production.

Problem statement

Low-resource clinics face major triage constraints: limited specialist access, high patient load, intermittent internet, and strict privacy requirements. Cloud-only AI tools are often impractical in these environments due to connectivity and infrastructure limitations.

MedAssist addresses this unmet need with a local-first clinical decision-support workflow that:

- structures patient intake,
- highlights urgent risk signals,
- standardizes escalation/referral decisions.

Target users are frontline clinicians and healthcare workers handling first-contact triage.

This system is not a diagnostic tool; it is designed to provide safer, faster clinical decision support.

Expected impact:

- improved triage consistency across staff and shifts,
- earlier escalation visibility for high-risk cases,
- clearer communication via structured output and patient-friendly explanation,
- practical deployment in privacy-sensitive, low-connectivity settings.

Overall solution

MedAssist is a human-centered triage application using Google HAI-DEF MedGemma as the clinical reasoning engine within a safety-first product pipeline.

HAI-DEF model usage:

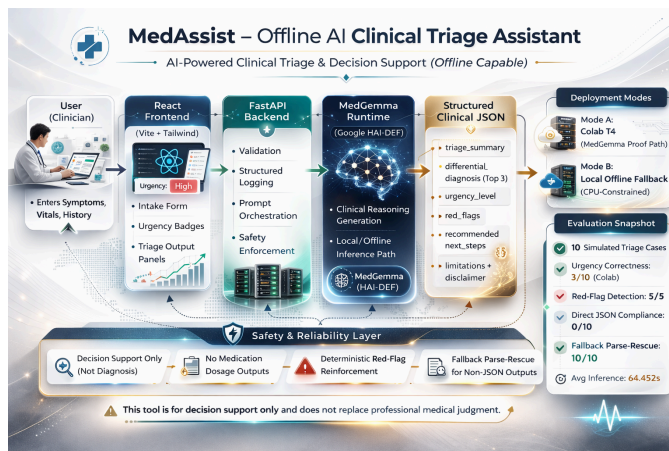
- MedGemma (`google/medgemma-4b-it`) is used to generate triage summaries, urgency reasoning, differential hypotheses, and recommended next steps.
- Runtime execution is demonstrated on Colab T4 GPU for reproducibility.
- Output hardening ensures schema-safe UI rendering under real-world model variability.

Why this is an effective HAI-DEF use:

- domain-specific clinical reasoning is applied to a real triage workflow,
- the submission demonstrates full product integration (not notebook-only inference),
- safety and deployment realism are treated as first-class requirements.

Technical details

Architecture:



Stack:

- Frontend: React (Vite), TailwindCSS, urgency badges, structured triage panels.
- Backend: FastAPI, Pydantic validation, structured logging, `.env` config.
- Runtime paths: Colab GPU MedGemma proof + local CPU fallback for constrained settings.
- Deployment: Docker-ready with reproducible setup instructions.

Safety implementation:

- mandatory disclaimer in UI and API outputs,
- no definitive diagnosis claims,
- no medication dosage recommendations,
- limitations section included in every response,

- deterministic reinforcement of critical red flags.

Evaluation setup:

- 10 simulated triage cases: fever, severe chest pain, stroke-like symptoms, abdominal pain, headache, dehydration, shortness of breath, pediatric fever, elderly confusion, trauma.
- Metrics: urgency correctness, red-flag detection hit-rate, direct JSON compliance, fallback parse-rescue, inference time.

Colab MedGemma T4 results:

- Total cases: 10
- Urgency correctness: 3/10 (30.00%)
- Red-flag detection hit-rate: 5/5 (100.00%)
- Direct JSON compliance: 0/10 (0.00%)
- Fallback parse-rescue: 10/10 (100.00%)
- Average inference time: 64.452s

Local backend production snapshot:

- Total cases: 10
- Urgency correctness: 7/10 (70.00%)
- Red-flag detection hit-rate: 5/5 (100.00%)
- Average inference time (fallback path): ~0.0s

Feasibility summary:

- end-to-end runnable product demo,
- reproducible MedGemma proof on Colab GPU,
- graceful degradation for constrained/offline deployments,
- practical trade-offs for real low-resource clinical environments.

Links:

- Repository: <https://github.com/RafayImraan/medassist.git>
- Colab notebook: https://colab.research.google.com/drive/1wGww4_SMsTDXAiZIXc33kYEkYluZIXJi?usp=sharing
- Video demo: <https://youtu.be/IUVYx0sIGJY>